

Encapsulation

📝 Notes by Menna Reda

قبل م نفهم يعني اي Encapsulation وبتتحقق في اي:

تعالى نشوف البرنامج بتاعنا دا

عندي class اسمه BankAccount

```
internal class BankAccount
{
    public decimal Balance;
}
```

اول حاجة فكرة اني اعمل class واعمل منه objects كل object منفصل ب data بتاعته ومفيش اي object يآثر على قيم object ثاني.. كل Object معمول له state خاص ومستقل عن الـ objects الثانية دا لوحدة نوع من الكبسلة “Encapsulation” -ال Class = Capsule تجمع الـ data والـ behavior المرتبطين ببعض.

بعد كد تعالى نشوف المشكلة اللي عندي عشان نعرف منها ال Encapsulation بتتم من خلال اي كمان

الرصيد المفروض له قواعد:

- مينفعش يبقى بالسالب.
- مينفعش أي حد يغيره عشوائيًا.
- السحب لازم يتأكد إن فيه رصيد كافي.
- الإيداع لازم يكون مبلغ موجب.

دلوقتي مين يقدر يـ access على الـ balance بتاع ال الاكونت دا اي حد يقدر يـ access ويحط اي قيمة في الـ balance زي كد:

```
BankAccount account = new BankAccount();
account.Balance = -1000;
```

مفيش حاجه تمنعة ان يحط قيم سالبة ممكن نقولي خلاص هعمل validation لو القيمة سالبة اخليها ب 0 مثلا عندي هنا في الـ main هقولك لا مش منطقي اصلا ان الـ main او الكلايت هو اللي يـ validate الداتا بتاعته اصلا مش مسؤوليته، اها... طب هتقولي مين قالك اصلا ان اي حد يقدر يوصل للـ Balance

هقولك من **public**

اي **public** دي: Public عبارة عن حاجه اسمها **access modifier** دي عبارة عن **keyword** بتحدد مين يقدر يـ access ال (class member) fields, methods بتاعتي جوا ال class و **public** تحديدًا معناها ان اي حد يقدر يـ access الحاجة دي اللي قبلها **Public**

اه معنى كد اننا عندنا انواع تانيه تحدد ان الـ Balance مش اي حد يغير في قيمته او يـ access عليه

اه عندنا انواع مختلفة بتحدد اي اللي اقدر أـ **access** عليه زي:

الـ **private**: محدش يقدر يوصل للحاجة اللي متعرفة **private** غير الـ **class members** بتاعتي يعني جوا ال class نفسه طب دي هوصلها ازاى برا الكلاس بق 🙄 : عن طريق اني اعمل حاجة وسيطة بيني وبين الحاجة دي **method** مثلا تاخد القيمة دي ومش تخزنها بس كد لا دي تـ **validate** عليها وتشف هل القيمة مسموح تتخزن ولا لا كمان اخليها ترجع ان القيمة دي متنفّش دخلي قيمة تانيه... اه دا انا كد خليت ان مش اي حد يتحكم في الحاجة بتاعتي ويحط اي **data** بالظبط كد.

الـ **protected**: متاح جوا ال class بتاعتي للـ **class member** بتوعي كمان متاح لاي حد يورث مني، اه يعني برودو مقدرش اوصل للحاجة الـ **protected** جوا ال main اه بالظبط كد اوصلها في ال class بتاعي او في class ثاني وارث مني “سواء كان ال class اللي وارث مني دا جوا او برا ال project بتاعي”.

الـ **internal**: قريبة للـ public ان اي حد يـ access على الحاجة بتاعتي بس الفرق انه لازم يكون في نفس الـ project بتاعي يعني لو class في project ثاني ساعتها لا ميقدرش يـ access عليها.

ودا الفرق بينها وبين public، الـ public اي حد جوا او برا الـ project بتاعي والـ internal اي حد بس اي حد جوا الـ project بتاعي بس.

كمان عندنا في الـ #C انواع ميكس زي:

الـ **protected internal**: معناها اي حد في نفس project واي حد وارث مني جوا او برا الـ project.

الـ **private protected**: معناها محدش يقدر يـ access غير انا واي حد وارث مني ولاكن بشرط الـ class الوارث مني يكون في نفس الـ project بتاعي يعني لو project ثاني مش هيشوفه.

نفس الـ project = نفس الـ assembly

بس كد دي هيا الـ access modifiers بتاعتي اللي تحددلي مين يقدر يـ access الحاجة بتاعتي ودا اللي بيحقق مبدأ الـ Encapsulation او التغليف انا بغلف او بخفي الحاجة بتاعتي.

اه يعني لو رجعنا للمشكلة بتاعتنا ثاني هتقولي سهلة بق هخلي الـ **balance private** بالظبط كد وهعمل methods هيا اللي تتحكم في الـ balance بتاعي بالظبط وتعالى نشوف ازاى:

الحاجات اللي محتاجها:

محتاج أعرف الرصيد

ومحتاج أعمل deposit و withdraw

فعمل الـ methods اللي تساعدني في الحاجات دي واخلبها public

```
class BankAccount
{
    private decimal balance;

    public void Deposit(decimal amount) // ايداع
    {
        if (amount > 0)
        {
            balance += amount;
        }
    }

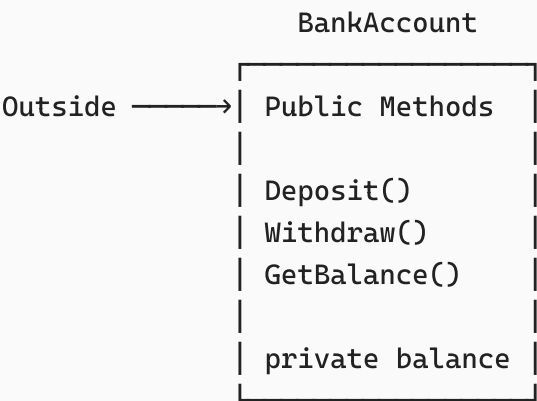
    public void Withdraw(decimal amount) // سحب
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
        }
    }

    public decimal GetBalance() // ترجعلي القيمة
    {
        return balance;
    }
}
```

```
BankAccount account = new BankAccount();
//account.balance = -1000; //Errorr ->'BankAccount.balance' is inaccessible due
to its
//protection level
```

```
account.Deposit(5000); // validation حتى لو ضفت قيمة غلط عندي
account.Withdraw(1000);

Console.WriteLine(account.GetBalance());
```



يمكن طبعا استخدم property تسهيلا للكود ولاكن دا مش موضوعنا

اللي مهم نطلع بيه إننا تحكمنا في الوصول للـ internal state بتاعتنا، ومنعنا التعديل المباشر عليها، وسمحنا بالتعامل معاها من خلال controlled operations.

وده جوهر الـ Encapsulation:

جمع البيانات والـ behavior المرتبط بيها داخل object، والتحكم في طريقة الوصول والتعديل على الـ internal state.

Summary:

Encapsulation:

It refers to the practice of binding data (fields) and the methods that operate on that data into a single unit.

How do we achieve it?

- private
- public
- protected
- internal
- protected internal
- private protected
- Properties
- Methods

Key Concepts

- Encapsulation hides the internal representation of an object and exposes only necessary operations.
- Fields are often kept private while access is provided through public properties or methods.
- It improves data security, code maintainability and flexibility.
- Access modifiers (private, public, protected, internal) control visibility of members.

Advantages

- **Data Protection:** Prevents unauthorized access to fields.
- **Controlled Access:** Exposes only required operations.

- **Code Flexibility:** Internal implementation can change without affecting external code.
- **Maintainability:** Reduces coupling between classes.